

Pintos Project 2-1 Design Document

```

+-----+
|           CS 140           |
| PROJECT 2: USER PROGRAMS |
|     DESIGN DOCUMENT     |
+-----+

```

---- GROUP ----

>> Fill in the names and email addresses of your group members.

Jeonghoon Park <hoonably@unist.ac.kr>
Deokhyeon Kim <ejrgus1404@unist.ac.kr>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

>> Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, lecture notes, and course staff.

ARGUMENT PASSING
=====

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

- Nothing added.

---- ALGORITHMS ----

>> A2: Briefly describe how you implemented argument parsing. How do you arrange for the elements of argv[] to be in the right order? How do you avoid overflowing the stack page?

- I modified process_execute function so that it passes only the program name to the thread_create function not the whole command including arguments.
- I added some code in the process_start function to parse arguments and push them in the stack. Below is the brief process of that code.
 1. Divide input command into a number of token separated by blanks, then store them in char * array.
 2. Push each arguments into the stack from right to left.
 3. Push padding (if required) to ensures that esp is multiples of 4.
 4. Push NULL sentinal since argv[argc] == NULL.
 5. Push address (in the stack) of each arguments from right to left.
 6. Push address (in the stack) of the first argument's address.
 7. Push argc.
 8. Push return address.

---- RATIONALE ----

>> A3: Why does Pintos implement strtok_r() but not strtok()?

- Since strtok() uses static variable in the internal implementation, it is unsafe in the multi-thread environment. However, since strtok_r() uses external variable instead of static variable it is safe in multi-thread environment.

>> A4: In Pintos, the kernel separates commands into a executable name and arguments. In Unix-like systems, the shell does this separation. Identify at least two advantages of the Unix approach.

- Since kernel doesn't have to consider parsing, kernel code is must simple.
- Kernel is much secure because program bugs related to parsing is restricted to user space (shell).
- Shell is more flexible, we can make many variations of shells to satisfy user requirements.

SYSTEM CALLS
=====

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

- struct lock file_lock; → Global lock for controlling concurrent file system access
- typedef int pid_t; → process ID type for exec and wait
- int fd_idx; → next file descriptor index
- struct file *fd_table[FD_MAX]; → file descriptor table (FD_MAX=128 setting now)

>> B2: Describe how file descriptors are associated with open files.
>> Are file descriptors unique within the entire OS or just within a
>> single process?

- Each thread (or process) manages its own file descriptors using a local fd_table array.
- File descriptors are not global. The descriptor number is valid only within the process and is independent of other processes.

---- ALGORITHMS ----

>> B3: Describe your code for reading and writing user data from the
>> kernel.

>> B4: Suppose a system call causes a full page (4,096 bytes) of data
>> to be copied from user space into the kernel. What is the least
>> and the greatest possible number of inspections of the page table
>> (e.g. calls to pagedir_get_page()) that might result? What about
>> for a system call that only copies 2 bytes of data? Is there room
>> for improvement in these numbers, and how much?

>> B5: Briefly describe your implementation of the "wait" system call
>> and how it interacts with process termination.

- In project 2-1, the process_wait() function was temporarily implemented using timer_msleep(3000) to simulate a blocking wait.

This was done only to allow related test cases to pass.

The function does not actually track or wait for child process termination.

A full implementation will be provided in project 2-2.

>> B6: Any access to user program memory at a user-specified address
>> can fail due to a bad pointer value. Such accesses must cause the
>> process to be terminated. System calls are fraught with such
>> accesses, e.g. a "write" system call requires reading the system
>> call number from the user stack, then each of the call's three
>> arguments, then an arbitrary amount of user memory, and any of
>> these can fail at any point. This poses a design and
>> error-handling problem: how do you best avoid obscuring the primary
>> function of code in a morass of error-handling? Furthermore, when
>> an error is detected, how do you ensure that all temporarily
>> allocated resources (locks, buffers, etc.) are freed? In a few
>> paragraphs, describe the strategy or strategies you adopted for
>> managing these issues. Give an example.

---- SYNCHRONIZATION ----

>> B7: The "exec" system call returns -1 if loading the new executable
>> fails, so it cannot return before the new executable has completed
>> loading. How does your code ensure this? How is the load
>> success/failure status passed back to the thread that calls "exec"?

>> B8: Consider parent process P with child process C. How do you
>> ensure proper synchronization and avoid race conditions when P
>> calls wait(C) before C exits? After C exits? How do you ensure
>> that all resources are freed in each case? How about when P
>> terminates without waiting, before C exits? After C exits? Are
>> there any special cases?

---- RATIONALE ----

>> B9: Why did you choose to implement access to user memory from the
>> kernel in the way that you did?

>> B10: What advantages or disadvantages can you see to your design
>> for file descriptors?

>> B11: The default tid_t to pid_t mapping is the identity mapping.
>> If you changed it, what advantages are there to your approach?

SURVEY QUESTIONS

=====

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want--these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

>> In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

>> Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

>> Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

>> Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

>> Any other comments?

==== Peer Evaluation ====

+-----+-----+	
Name	Contribution
+-----+-----+	
Jeonghoon Park	50%, Implemented system calls
+-----+-----+	
Deokhyeon Kim	50%, Implemented argument passing
+-----+-----+	